Context Engineering in Practice

Building Agents That Don't Forget (or Remember Too Much)

Matt Redmond

17 Mar 2026

Today's Journey

The guest lecture established the **theory**. This session is a **live demo**: six progressive improvements to the same agent, all measured with the same eval.

1. Context Window as Working Memory
   Budget, not buffer
2. Seeing What Your Agent Sees
   Instrument first, optimize second
3. The Practitioner's Toolkit
   Three patterns that actually work
4. Live Demo: Demos 01–06
   Context shrinks; accuracy mostly improves
5. Recursive Language Models
   The model manages its own context
6. Wrap-Up

Outline

1. The Context Window as Working Memory
2. Seeing What Your Agent Sees
3. The Practitioner's Toolkit
4. Live Demo: Six Progressive Approaches
5. Recursive Language Models
6. Wrap-Up

Bridging from the Theory

The guest lecture mapped the landscape:

The three-layer hierarchy:

1. **Prompt engineering** — the right words for one call
2. **Context engineering** — the right information for an agent over time
3. **Harness engineering** — the scaffold, orchestration, tooling

The failure modes:

- Context **poisoning** — bad data directs behavior
- Context **distraction** — irrelevant data dilutes attention
- Context **confusion** — contradictory data paralyzes reasoning
- **Fighting the weights** — context conflicts with learned priors

Today's Question

What does good context management actually look like in code?

We will build up the answer through a working demo.

The Working Memory Analogy

Human Working Memory

- Capacity: 7 ± 2 chunks (Miller, 1956)
- Degrades under cognitive load
- Recent and salient items dominate
- Irrelevant information causes interference
- Offloads to long-term memory or external notes

Analogy

You can remember your childhood best friend's phone number forever. You cannot juggle 20 phone numbers simultaneously.

Recap: LLM Context Window

- Capacity: hundreds of thousands to about a million tokens
- Effective attention degrades over very long contexts
- Recent tokens (empirically) carry higher effective weight
- Irrelevant content causes distraction and confusion
- Offloads to tools, memory stores, or sub-agents can help

Key Point

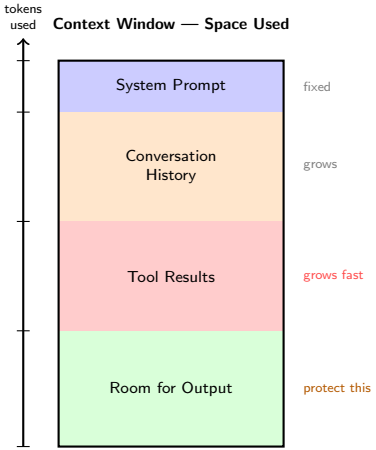
More information is *not always better*. A **focused, well-curated** context outperforms a bloated one.

Context as a Budget

Every token appended is a spending decision.

The context window is not an infinite scratchpad—it is a **shared resource** that every part of the agent pipeline must compete for.
Programming analogy: context window is somewhat like memory allocated on the stack. You can push new stack frames by calling subagents, etc.

- System prompt sets the base cost
- Each tool call adds input *and* output tokens
- Long conversation history compounds quickly
- The output you actually want needs room too



A Practitioner's Framing

Ask of every piece of content you add:

"Does this **earn its spot** *in the context?"*

Outline

1. The Context Window as Working Memory
2. Seeing What Your Agent Sees
3. The Practitioner's Toolkit
4. Live Demo: Six Progressive Approaches
5. Recursive Language Models
6. Wrap-Up

Instrument Before You Optimize

Many people never look at what they are actually putting in the context.

This is like optimizing database performance without looking at EXPLAIN query plans.

A simple way to inspect the context window is invaluable. Here's a demo impl:

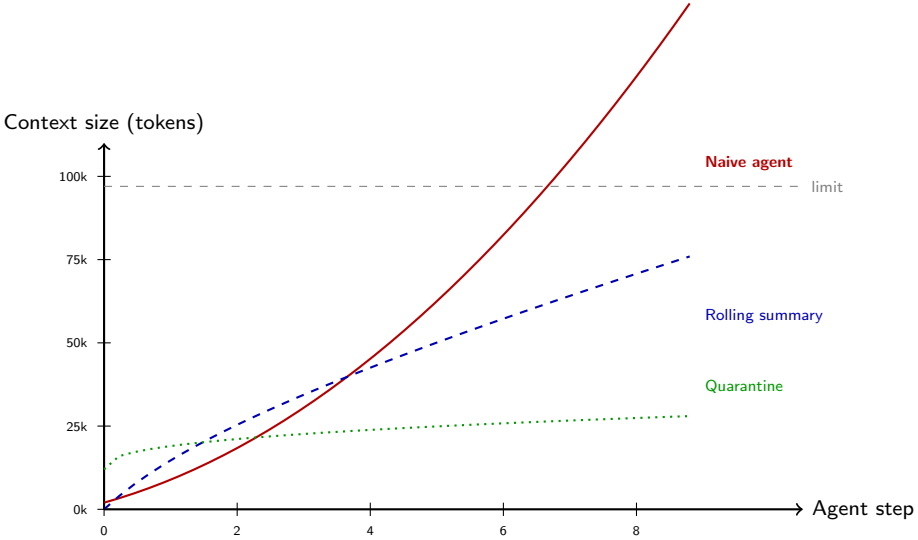
```python
import tiktoken

def inspect_context(
    messages: list[dict[str, str]]
) -> None:
    enc = tiktoken.encoding_for_model("gpt-4o")
    total = 0
    for msg in messages:
        content = msg.get("content") or ""
        n = len(enc.encode(content))
        total += n
        role = msg["role"].ljust(12)
        bar = "=" * (n // 50)
        print(f"{role} {n:>5} tok  |{bar}")
    print(f"{'TOTAL':>12} {total:>5} tok")
```

What You Will Learn to See

- Which message types dominate
- How fast the context grows per step
- Whether tool result sizes are proportional to their usefulness
- When you are approaching the limit

The Context Growth Curve



Outline

1. The Context Window as Working Memory
2. Seeing What Your Agent Sees
3. The Practitioner's Toolkit
4. Live Demo: Six Progressive Approaches
5. Recursive Language Models
6. Wrap-Up

Three Patterns That Actually Work

1. Rolling Summarization

After each step, compress accumulated content into a compact running summary.

Good for: long sequential tasks where each step builds on the last.

Cost: one extra LLM call (summarization) per step.

2. Context Quarantining

Isolate each sub-task in its own context thread, via a "sub-agent" somehow. Return only the distilled output to the main context.

Good for: parallel, independent sub-tasks.

Cost: extra API calls; harder to share state. Handoff to sub-agent can miss critical info.

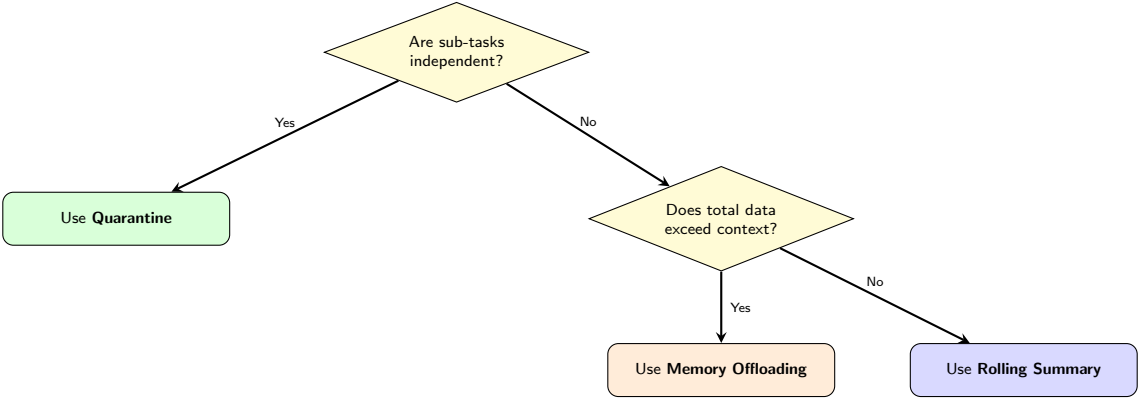
3. Memory Offloading

Write intermediate findings to an external store. Read back only what is currently needed.

Good for: long-horizon agents where total data exceeds any context limit.

Cost: retrieval latency; retrieval errors.

Choosing the Right Pattern



In Practice

Most production compound agent systems combine all three. Sub-agents for parallel steps; rolling summary for sequential depth; offloading for persistence across sessions.

Outline

1. The Context Window as Working Memory
2. Seeing What Your Agent Sees
3. The Practitioner's Toolkit
4. Live Demo: Six Progressive Approaches
5. Recursive Language Models
6. Wrap-Up

The Scenario

One eval, six approaches

Corpus: 400 Axiom Software support tickets (pipe-delimited).

```
id=T001 || plan=Enterprise || date=...  || msg=...
```

Task: answer 15 classification questions about the corpus. Questions require *semantic* reasoning — keyword matching fails.

Eval metric: score (0–1 per question, proportional to error) + peak context tokens.
Both *should* improve together as context management gets better, but let's see!

Files in `longform_demo/`:

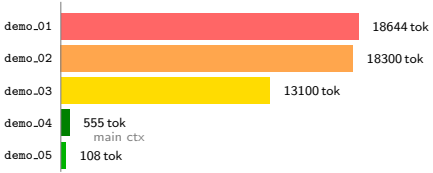
`demo_01_baseline.py` `demo_02_prompt_trim.py`
`demo_03_prune_results.py` `demo_04_summarize.py`

`demo_05_quarantine.py` `demo_06_rlm.py`

The Key Insight

The corpus is the same throughout. What changes is how much of it ends up in any single LLM call.

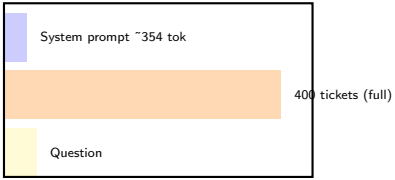
Peak context per LLM call:



Demo 01 — The Broken Baseline

What is wrong:

- System prompt: many tokens of padded, redundant prose
- All tickets dumped verbatim into every call
- Each call sees the same irrelevant noise regardless of question



demo_01_baseline.py — key loop

```
for q in QUESTIONS:
    messages = [
        {"role": "system", "content": SYSTEM}, # bloated
        {"role": "user", "content":
            f"Tickets:\n\n{CORPUS}\n\n"          # full dump
            f"Question: {q['question']}"},
    ]
    peak = inspect_context(messages)  # ~18,589 tok
```

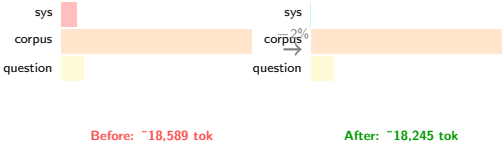
Failure modes

Distraction — tickets unrelated to the question compete for attention.
Lost-in-middle — LLMs under-weight tokens far from the ends.

Run: `uv run demo_01_baseline.py`
Result: Score **5.94/15** Peak **18,644 tok**

Demo 02 — System Prompt Compression

Technique: rewrite the system prompt as a single imperative line. Same behaviour; ≈97% fewer prompt tokens — paid on *every* call.



BEFORE – 354 tok

```
SYSTEM = """You are a highly experienced,
professional, and thorough support-ticket
analyst... [8 more padded paragraphs]
...Accuracy is paramount for business
decisions. Always ensure your answer is
correct and complete before responding."""
```

AFTER – 10 tok, same behaviour

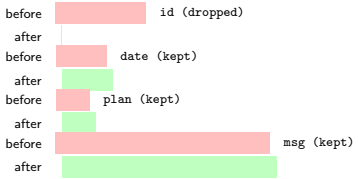
```
SYSTEM = (
    "Count matching tickets. "
    "Reply with a single integer."
)
```

Rule of Thumb

If the sentence starts with "Remember that..." or "Always ensure...", it can be cut. System prompts shrink >97% when written as instructions, not essays. Score improves: **6.92/15** vs 5.94/15 baseline.

Demo 03 — Ticket Field Pruning

Technique: strip each ticket to only the fields needed. If you know ahead of time what questions are getting asked, you can prune the corpus to just the minimal set of fields needed to answer each question. Keep `plan` (for Q4, Q6, Q7, Q10–12) and `msg` (for Q1–3, Q5, Q8–9, Q11–15). Drop `id` — it earns no spot in the context. Keep `date` only for date-range questions (Q13).



BEFORE – full record

```
# id=T001 || plan=Enterprise ||
# date=2024-11-01 || msg=After our...
```

AFTER – relevant fields only

```python
def _prune_line(line: str) -> str:
    plan = re.search(r"plan=(\w+)", line)
    msg  = re.search(r"msg=(.+)$",  line)
    return (f"plan={plan.group(1)} || "
            f"msg={msg.group(1)}")
# ~30% fewer tokens, same semantic content
```

Key Question

For each field: could the model answer this question without it? If yes — drop it.

Result: corpus drops from ∼18,589 to ∼13,047 tok (−30%). Score improves to **8.78/15** — less noise helps.

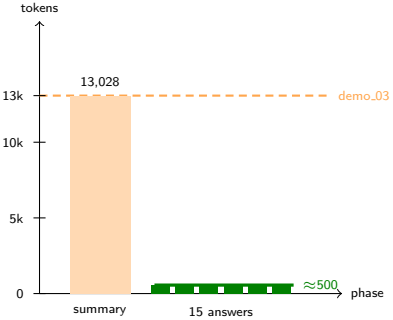
Demo 04 — Summarize then Answer

Intuition: compress the corpus into a summary first, then answer each question cheaply from the summary. Less data per call!

demo_04_summarize.py

```
# Step 1: one call, read all tickets
summary, _ = build_summary(client)
# summary call = 13,028 tok (once)

# Step 2: answer each question from summary
for q in QUESTIONS:
    raw = answer_from_summary(
        client, summary, q["question"])
# answer call = ~500 tok  (tiny!)
```



The Catch

Summaries are **lossy**. A 300-word brief captures "≈20% auth tickets" — not the exact count of 100. Exact counting tasks *require* access to the raw data.

Result

Answer calls: ≈**500 tok** (97% smaller).
But score *drops* to **4.38/15** — worse than baseline!
Lesson: don't summarise data you need to count exactly.

Demo 05 — Question Quarantine

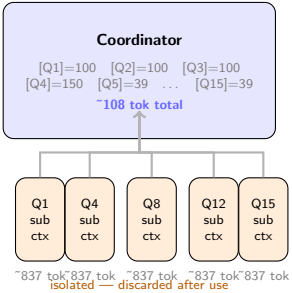
Technique: each question is answered inside its own isolated sub-context using batches of 20 tickets. The coordinator never sees ticket data — only integers. Sub-contexts are discarded after each question.

BEFORE – question loop touches ticket data

```
for q in QUESTIONS:
    messages = [sys,
        {"content": f"{CORPUS}\n\n{question}"}]
    # coordinator context grows with corpus
```

AFTER – coordinator sees only integers

```
for q in QUESTIONS:
    raw = quarantined_answer(client, q)
    # sub-context: batches of 20 tickets
    # each sub-ctx discarded after use
    main_messages.append(
        {"content": f"[{q['id']}] = {raw}"})
# coordinator context: ~108 tok (15 integers)
```



Key Insight

Coordinator context bounded by # answers, not corpus size. Score: **8.80/15** (best harness result). Sub-ctx peak: **837 tok** vs 18,644 baseline.

The Full Picture

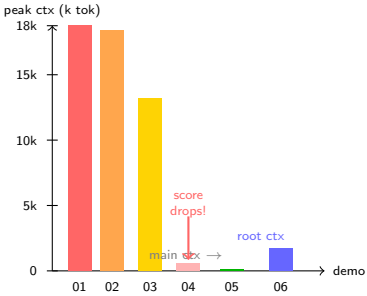
Same corpus. Same 15 questions. Six approaches.

	Technique	Ctx (peak)	Score /15
01	Baseline	18,644 tok	5.94
02	+ Prompt compression	18,300 tok	6.92
03	+ Field pruning	13,100 tok	8.78
04	Summarize then Answer	555 tok*	4.38
05	+ Quarantine	108 tok†	8.80
06	RLM	1,733 tok‡	9.15

*answer calls; summary call 13,028 tok

†coordinator; sub-ctx ≈837 tok (−96% vs 01)

‡peak root ctx; avg 1,733 tok; 91% savings vs. naive



Key observations

Demo 04 (summarize) shrinks context but *hurts* accuracy.
Demo 05 (quarantine) achieves both goals simultaneously.
Demo 06 (RLM) *exceeds* naive accuracy at 91% less context.

Comparing the Approaches

	Peak context	API calls	Parallelizable	Isolation
Naive (demo_01–03)	$O(C)$	Q	Limited	None
Summarize (demo_04)	$O(C) + O(S)$	$1 + Q$	No	None
Quarantine (demo_05)	$O(C/B)$	$Q \cdot B$	Yes	Per question
RLM (demo_06)	$O(\mathrm{I/O})$	dynamic	Yes	Per sub-LLM

C = corpus size, Q = questions, B = batches, S = summary size, I/O = REPL output only

Quarantine wins when:

- Sub-tasks are fully independent (no shared state)
- You need exact answers from raw data
- Cross-task contamination is a real concern
- You want to parallelize for latency

Summarize when:

- Approximate answers are acceptable
- Questions are about themes, not exact counts
- Downstream tasks don't require precision

Watch out

Summarisation lost 26% accuracy here.

Outline

1. The Context Window as Working Memory
2. Seeing What Your Agent Sees
3. The Practitioner's Toolkit
4. Live Demo: Six Progressive Approaches
5. Recursive Language Models
6. Wrap-Up

Beyond Harness Engineering: The Model Manages Its Own Context

Demos 01–05 show the *harness* managing context on behalf of the model: we prune, summarise, quarantine.

The next step: give the model a Python REPL and let it manage its own context through code.

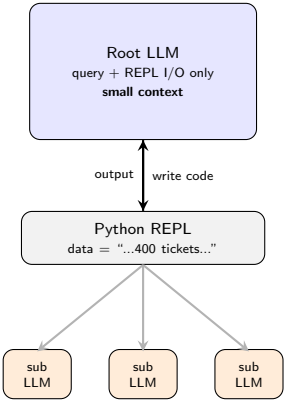
Recursive Language Model (RLM)

Zhang & Khattab, Oct 2025 (arXiv:2512.24601)

The root LLM only sees the **query** and **REPL I/O**. The full corpus is a Python variable it can inspect programmatically. It spawns **isolated sub-LLMs** to process chunks — their raw output never pollutes the root's context.

Key result (paper): RLM(GPT-5-mini) outperforms GPT-5 on the OOLONG long-context benchmark by >**114%** at roughly the same cost per query.

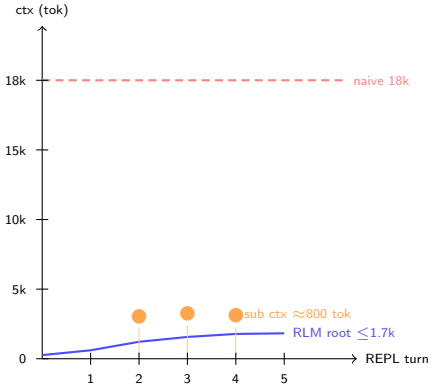
Our demo (gpt-4o-mini): exceeds naive accuracy (9.2/15 vs 6.9/15) with **91% context savings** (avg root ctx 1,733 tok vs 18,248 tok naive).



What the RLM Does: Emergent Strategies

Given only a query and a large variable in memory, models spontaneously develop structured strategies:

1. **Peek** — inspect structure before committing
   ```
   print(data[:400])
   ```
2. **Grep** — filter to a relevant subset
   ```
   rows = [l for l in data.splitlines()
       if "payment" in l.lower()]
   ```
3. **Partition + Map** — chunk and batch to sub-LLMs
   ```
   results = llm_batch(chunks)
   ```
4. **Aggregate in Python** — count, sum, filter
   ```
   answer["content"] = str(sum(results))
   ```
5. **Iterate** — if uncertain, refine and re-query



Key Property

Sub-LLM contexts spike and vanish. The root LLM's context grows only with REPL I/O, not with raw data. **91% savings** measured in our eval.

Demo 06: OOLONG-Style Eval on 400 Tickets

Corpus: 400 synthetic Axiom support tickets

```
id=T001 || plan=Enterprise || date=...  || msg=...
```

Why this is hard for a naive single call:

- Questions require *semantic* classification of free-text
- Categories not detectable by keyword matching alone: "session timeouts" → auth; "amount doesn't add up" → billing
- Loading all 400 tickets uses ≈18.6k tokens before the model writes a single word

Fifteen questions (ground truth known):

- Q1: auth/access problems? *(100)* Q2: billing? *(100)*
- Q3: pipeline failures? *(100)* Q4: Enterprise plan? *(150)*
- Q5: urgent/production-blocking? *(39)*
- Q6: Starter plan? *(98)* Q7: Team plan? *(152)*
- Q8: performance issues? *(50)* Q9: feature requests? *(50)*
- Q10–Q15: cross-filter, date, and semantic questions...

```
cd longform_demo/
uv run demo_06_rlm.py

# full eval (all 15 questions):
uv run demo_06_rlm.py --eval
```

The ASCII visualiser prints the RLM's full trace: each REPL turn, code written, sub-LLM calls dispatched, and root-context size after each step.

Results from `--eval` run

Naive: avg ctx 18,248 tok score 6.90/15
RLM: avg root ctx 1,733 tok score 9.15/15
Context savings: 91% — *better* accuracy, much smaller root context.
Peak root ctx: 2,605 tok (vs 18,312 tok naive peak).

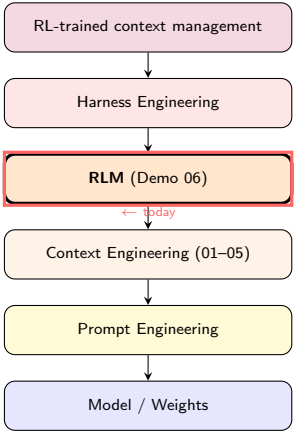
The Full Hierarchy

We have now walked the entire stack:

Demo	Technique	Who decides?
01–02	Prompt trim	You
03	Field pruning	You
04	Summarisation*	You
05	Quarantine	You
06	**RLM**	**The model**

*Demo 04 is the "bad" approach — intentional.

Each demo delegates more context-management responsibility to the model itself.



The Research Direction (2026)

Train models *end-to-end* via RL so that peek/grep/partition become learned behaviours, not prompted heuristics.

Prime Intellect RLM blog (Jan 2026): `primeintellect.ai/blog/rlm`

Outline

1. The Context Window as Working Memory
2. Seeing What Your Agent Sees
3. The Practitioner's Toolkit
4. Live Demo: Six Progressive Approaches
5. Recursive Language Models
6. Wrap-Up

Key Takeaways

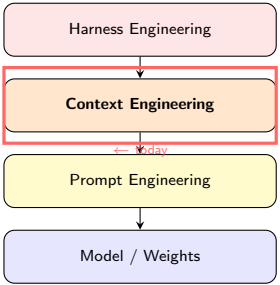
1. **Measure first.** You cannot manage what you cannot see. A simple token counter changes how you think about your agent.
2. **Every append is a budget decision.** Ask whether each piece of content *earns its spot*. Raw tool output rarely does at full length.
3. **The right technique depends on task structure.**
4. **Context management is infrastructure.** In a mature agent system it lives in the harness, not the task logic. The patterns you applied manually today become middleware.
5. **The frontier: let the model manage itself.** RLMs show that with a REPL and sub-LLMs, the model can outperform approaches that load entire corpora into context.

The Bigger Picture

Today's exercises operated at the **context engineering** layer.

As systems scale, these patterns become part of the **harness**:

- Summarization middleware sits between tool calls and the message list
- Quarantine is the default mode for sub-agent invocation
- Memory offloading becomes a retrieval system (you already built one in Units 6–7)
- RLMs point toward models that learn all of this end-to-end



The Arc

Units 1–5: foundations and prompt engineering
Units 6–7: retrieval
Unit 8: context management
Units 9–10: evaluation at scale
Unit 11: data pipelines
Units 12+: production systems

Next Up

Evaluation — how do we know whether our context management is actually working?